

Propeller–Wing case setup in OpenFOAM

Toran Bahadur Niroula, Yukesh Karki, and Chandan Bose

July 8, 2026

Contents

1	Introduction	3
1.1	Objective	3
1.2	Scope	4
2	Geometry and Domain Setup	5
2.1	Geometry Preparation	5
2.2	Computational Domain	6
3	Meshing	8
3.1	Base Mesh Generation	8
3.2	snappyHexMesh	10
3.2.1	Refinement Surfaces and Regions	13
3.2.2	Layer Addition	15
3.3	Creating AMI Patches	15
3.3.1	createPatchDict Configuration	16
3.3.2	Running createPatch	17
3.4	Mesh Quality Checks	17
4	MRF Setup	20
4.1	Multi Reference Frame	20
4.1.1	MRFProperties configuration	20
5	Dynamic Mesh and AMI Setup	22
5.1	Arbitrary Mesh Interface	22
5.2	dynamicMeshDict Configuration	22
6	Boundary and Initial Conditions	25
6.1	Case Initialization	25
6.1.1	Velocity Field (U)	25
6.1.2	Pressure Field (p)	26
6.1.3	Turbulence Fields	27
6.2	Properties and Constants	29
7	Solver Configuration and Controls	31
7.1	Steady state Simulation	31
7.1.1	controlDict configuration	31
7.2	Unsteady Simulation	32
7.2.1	controlDict configuration	32
7.3	Discretization Schemes	33

7.3.1	fvSchemes configuration	33
7.4	Linear Solvers	34
7.4.1	fvSolution configuration	35
7.4.2	PIMPLE Controls	36
7.4.3	SIMPLE Controls	37
8	Function Objects and Data Extraction	38
8.1	Force and Moment Extraction	38
8.2	Monitoring the execution	40
9	Appendices	41
9.1	mesh generation script	41
9.2	steady state run script	41
9.3	unsteady run script	41

Chapter 1

Introduction

1.1 Objective

OpenFOAM is one of the most widely used free and open source CFD software packages. Despite its popularity, the learning curve can be quite steep for first time users. Dynamic mesh cases, in particular, are additionally challenging due to complex mesh motion handling, stricter mesh quality requirements, and the proper configuration at interface boundaries. This documentation aims to provide a detailed walkthrough for setting up and running a propeller-wing simulation in OpenFOAM. The sole objective of this documentation is to introduce users to the key steps required to setup a robust dynamic case using the Arbitrary Mesh Interface (AMI) method, in which a rotating propeller operates in front of a stationary wing.

This guide aims to serve as a technical instructional reference for users developing similar cases involving rotating geometries or other AMI-based setups.

Goals of this documentation:

- To guide readers through the creation of a physically consistent dynamic simulation involving a rotating propeller and a stationary wing.
- To explain setup procedure of AMI in handling mesh motion between rotating and stationary domains.
- To document and discuss common challenges encountered in dynamic simulations such as poor mesh quality, or AMI zero weight issues and methods to overcome them.
- To provide a step by step explanation of essential configuration files, including `dynamicMeshDict`, `snappyHexMeshDict`, and function objects for force and propeller data extraction.

This document assumes that the reader has knowledge of basic file structure and dictionaries and experience of setting a case in OpenFOAM. This case was set up in keysight OpenCFD release of OpenFOAM.

1.2 Scope

This documentation focuses on the setup and configuration of dynamic propeller-wing simulations or similar case in OpenFOAM. The emphasis here is on how to successfully setup a physically consistent case. Numerical performance of setup, interpretation and post-processing of obtained result and experimental validation are secondary or outside the scope of this documentation. The following key aspects are focused here:

- Setup of steady state and transient propeller-wing simulations.
- setting up steady-state propeller-wing case with MRF approach.
- Dynamic mesh handling using the Arbitrary Mesh Interface (AMI) approach.
- Strategies for good quality mesh generation and mesh refinements for this or similar cases.
- Monitoring convergence behavior during transient simulations.
- Extraction of aerodynamic forces, coefficients, and propeller performance data.

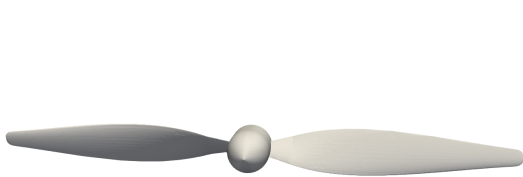
This document is solely focused on setting the case. It doesn't intend to benchmark performance or validate the results. Instead, it aims to provide a technically sound reference for setting and running reliable sliding mesh case relevant for aerodynamics studies.

Chapter 2

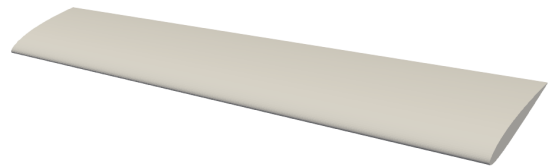
Geometry and Domain Setup

2.1 Geometry Preparation

The first step of a CFD simulation is the preparation of the geometry, which represents the physical components around or through which the flow develops. In this study, the required geometries are the APC 12×6EP two-bladed propeller (diameter 304.8 mm) and a rectangular wing with NACA 2412 airfoil, half-span 1450 mm, and chord 236 mm. The primary reason for the study of this propeller and wing is because they are widely used in small-sized UAV the 10–30 kg take-off mass range. Additionally, performance data for the APC 12×6EP propeller are publicly available, allowing comparison between manufacturer data and CFD predictions. The propeller geometry was constructed using CAD software by reconstructing the blade from the station-wise geometric data provided by APC. The dataset includes spanwise distributions of chord, twist, and sectional airfoil shapes. The corresponding airfoil coordinate files were obtained from online airfoil databases such as airfoiltools or the UIUC Airfoil Coordinates Database. Blade lofting and twist application were automated through Python scripts within the CAD environment to ensure consistent parameterization and reduce manual modeling errors. The wing was generated as a rectangular planform with a NACA 2412 profile. The airfoil coordinates were scaled to the desired chord, imported into the CAD software, and extruded to the intended span suitable for simulation. After generating the STL files, their quality was examined by tools like surfaceCheck available in openFoam. If there are any serious issues with geometry like presence of unintended holes on the surface, non-manifold edges and faces, zero area faces, etc, it can be repaired with tools like meshLab or Blender. Any required repositioning, rotation, or scaling was performed using OpenFOAM’s surfaceTransform-Points, ensuring the propeller axis was aligned with the global axis system and that the wing was positioned in the appropriate aerodynamic reference frame.



(a) geometry of APC1260EP propeller



(b) geometry of NACA2412 rectangular wing

Figure 2.1: wing and propeller geometry used for this simulation

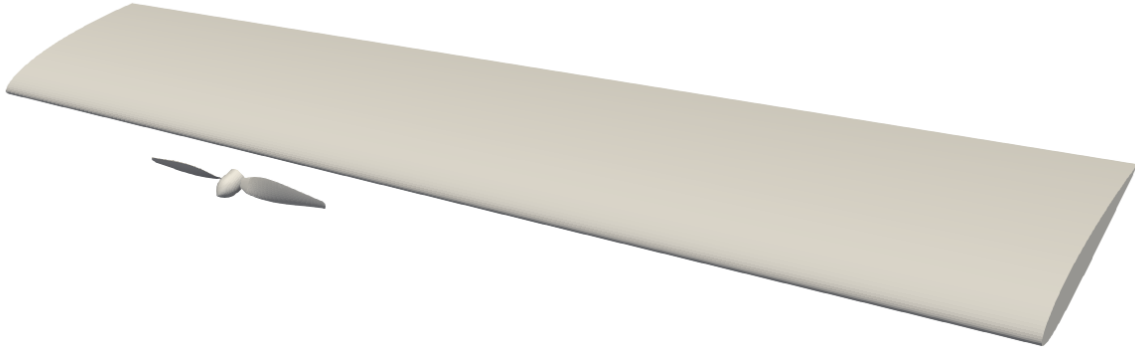


Figure 2.2: wing and propeller geometry in their respective position

2.2 Computational Domain

The computational domain for the simulation was designed to ensure that the flow field around the propeller–wing configuration develops without significant interference from domain boundaries. The domain dimensions are 12 m (x–direction), 6 m (y–direction), and 3 m (z–direction). These lengths correspond to approximately:

- 40, 20, and 10 times the propeller diameter along the x, y, and z directions respectively.
- 50, 25, and 12.5 times the wing chord in the same directions.

The origin of the coordinate system is placed at the wing leading edge on the symmetry plane. The upstream inlet boundary is located 4 m upstream of the origin, while the outlet is 8 m downstream, following the freestream x-direction. The wing root lies on the symmetry plane ($z = 0$), with the leading edge passing through the origin. The propeller hub center is positioned 0.54 m in the +z direction (span direction), 0.05 m upstream of the wing leading edge, with its rotation axis aligned parallel to the x-axis, consistent with tractor configuration propellers. The use of mirror symmetry is justified because the simulation represents a twin-propeller aircraft in straight, zero–sideslip flight. The off-center propeller location and the absence of lateral flow allow a half-domain approach, significantly reducing computational cost. This setup provides a sufficiently large buffer region for both propeller-induced flow and the wing wake, ensuring numerical stability and minimizing boundary interference.

Table 2.1: Computational domain boundaries and component positions.

Item	Position
Domain Boundaries	
Inlet plane	$x = -4$ m
Outlet plane	$x = +8$ m
Symmetry plane	$z = 0$ m
Spanwise far-field	$z = +3$ m
Top far-field	$y = +3$ m
Bottom far-field	$y = -3$ m
Component Positions	
Wing root	$z = 0$
Wing tip	$z = 1.45$ m
Wing leading edge	$(0, 0, z)$ m
Wing trailing edge	$(0.236, 0, z)$ m
Propeller hub center	$(-0.05, 0, 0.54)$ m
Propeller axis	Parallel to x -axis

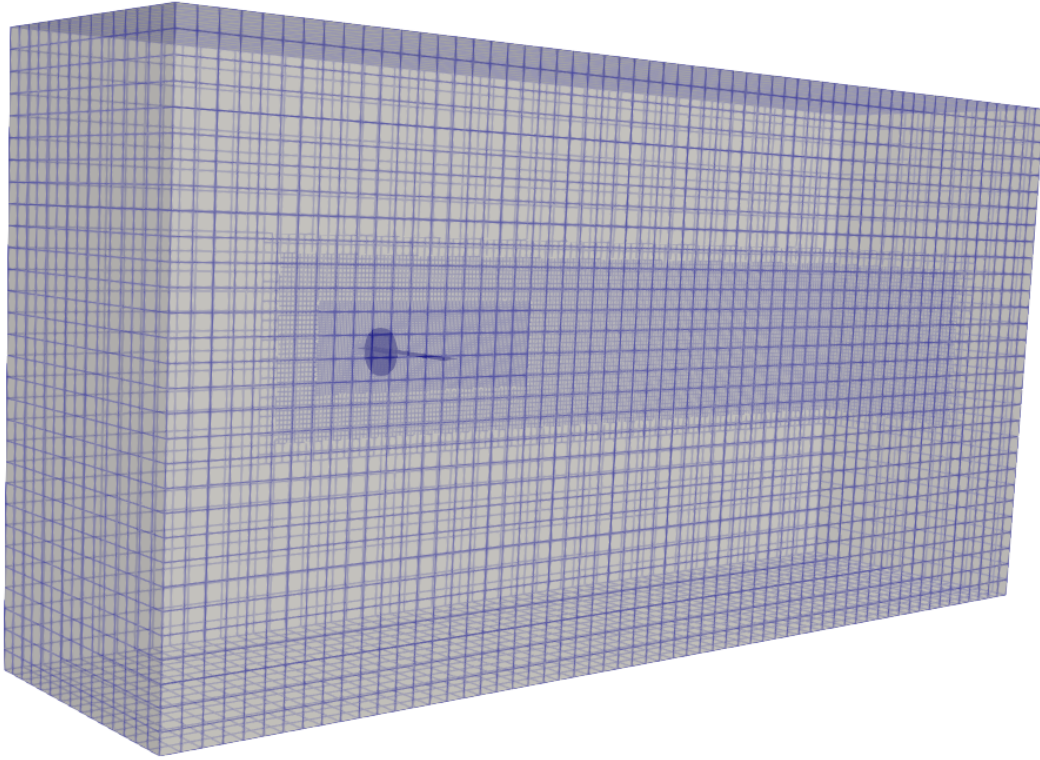


Figure 2.3: The computational domain for this setup

Chapter 3

Meshing

3.1 Base Mesh Generation

The background mesh for the simulation was generated using the `blockMesh` utility in OpenFOAM. The background base mesh provides a structured hexahedral grid on which `snappyHexMesh` can perform local refinement around the geometry file specified: propeller and the wing in our case. The base mesh must be coarse enough for computational efficiency while still sufficiently resolved to support stable refinement, accurate geometry capturing, and generation of fine layers near boundary. A single hexahedral block was used to discretize the entire computational domain. The selected resolution of $(60 \times 30 \times 15)$ cells in the x -, y -, and z -directions corresponds to an approximate background cell size of 0.2 m. Uniform grading was selected in all directions (i.e., `simpleGrading (1 1 1)`) as the additional localized refinement is handled entirely by `snappyHexMesh`. This approach is simple and prevents unnecessary mesh density in far-field regions while maintaining high resolution near the propeller and wing.

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       blockMeshDict;
}
scale          1;
vertices (
    (-4 -3 0)    // 0
    ( 8 -3 0)    // 1
    ( 8  3 0)    // 2
    (-4  3 0)    // 3
    (-4 -3 3)    // 4
    ( 8 -3 3)    // 5
    ( 8  3 3)    // 6
    (-4  3 3)    // 7
);
blocks
(
    hex (0 1 2 3 4 5 6 7) (60 30 15) simpleGrading (1 1 1)
);
boundary
(
    farField
```

```

{
    type patch;
    faces
    (
        (4 5 6 7)
        (3 7 6 2)
        (1 5 4 0)
    );
}
inlet
{
    type    patch;
    faces    ((0 4 7 3));
}
outlet
{
    type    patch;
    faces    ((2 6 5 1));
}
symmetry
{
    type    symmetryPlane;
    faces    ((0 3 2 1));
}
);

```

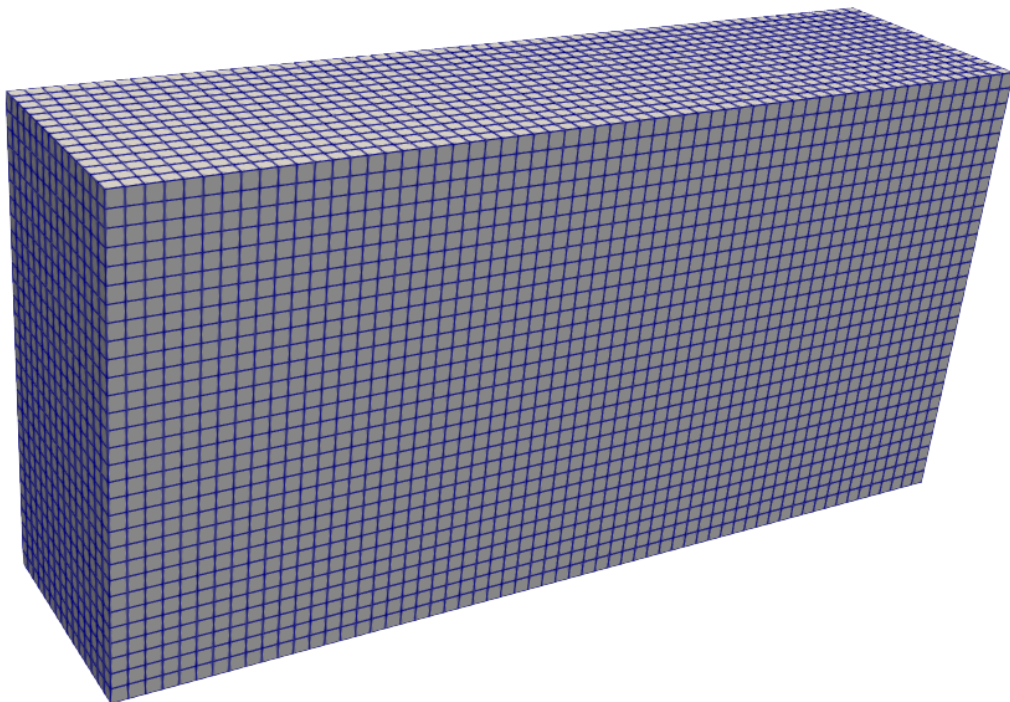


Figure 3.1: Base mesh generated by blockMesh

3.2 snappyHexMesh

`snappyHexMesh` was used to generate the final mesh for the simulation. It takes the base mesh and tessellated geometry files and performs local refinement, surface snapping, and boundary-layer meshing. The three main stages of the `snappyHexMesh` meshing algorithm are:

1. **Castellated mesh generation** – background mesh refinement and region-based cell removal.
2. **Surface snapping** – snapping castellated mesh vertices onto geometry surfaces.
3. **Layer addition** – extrusion of boundary-layer cells near solid surfaces.

Layer addition is especially important for capturing near-wall viscous effects, particularly in turbulent flows. Each stage can be toggled individually in the `snappyHexMeshDict`.

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       snappyHexMeshDict;
}
castellatedMesh true;
snap           true;
addLayers      true;
geometry
{
    propeller.stl
    {
        type triSurfaceMesh;
        name propeller;
    }
    wing.stl
    {
        type triSurfaceMesh;
        name wing;
    }
    ami
    {
        type searchableCylinder;
        point1 (-0.09 0 0.54);
        point2 (-0.03 0 0.54);
        radius 0.25;
    }
    box1
    {
        type box;
        min   (-1 -1 0);
        max   (8 1 2);
    }

    box2
    {
        type box;
        min   (-0.5 -0.5 0);
    }
}
```

```

        max (2.5 0.5 1.5);
    }
    box3
    {
        type box;
        min (-0.085 -0.025 0.365);
        max (-0.035 0.025 0.715);
    }
    propWake
    {
        type searchableCone;
        point1 (-0.03 0 0.54);
        radius1 0.25;
        innerRadius1 0;
        point2 (1 0 0.54);
        radius2 0.3;
        innerRadius1 0;
    }
}
castellatedMeshControls
{
    maxLocalCells 8000000;
    maxGlobalCells 12000000;
    minRefinementCells 1;
    maxLoadUnbalance 0.10;
    nCellsBetweenLevels 2;
    features
    (
        { file "wing.eMesh"; level 6;}
        { file "propeller.eMesh"; level 9;}
    );
    refinementSurfaces
    {
        propeller
        {
            level (8 9);
            patchInfo
            {
                type wall;
                inGroups (propGroup);
            }
        }
        wing
        {
            level (7 7);
            patchInfo
            {
                type wall;
                inGroups (wingGroup);
            }
        }
        ami
        {
            level (5 5);
            faceType baffle;
            cellZone rotor;
            faceZone ami;
            cellZoneInside inside;
        }
    }
}

```

```

    }
}
resolveFeatureAngle 30;
refinementRegions
{
    ami
    {
        mode    inside;
        levels   ((1E15 5));
    }
    box1
    {
        mode    inside;
        levels   ((1E15 2));
    }
    box2
    {
        mode    inside;
        levels   ((1E15 3));
    }
    box3
    {
        mode    inside;
        levels   ((1E15 6));
    }
    propWake
    {
        mode    inside;
        levels   ((1E15 5));
    }
}
locationInMesh (5 0 1.5);
allowFreeStandingZoneFaces false;
}
snapControls
{
    nSmoothPatch 8;
    tolerance 1.5;
    nSolveIter 100;
    nRelaxIter 6;
    nFeatureSnapIter 10;
    implicitFeatureSnap true;
    explicitFeatureSnap false;
    multiRegionFeatureSnap true;
}
addLayersControls
{
    relativeSizes false;
    layers
    {
        "propeller"
        {
            nSurfaceLayers 5;
        }
        "wing"
        {
            nSurfaceLayers 6;
        }
    }
}

```

```

    "ami"
    {
        nSurfaceLayers 0;
    }
    "ami_slave"
    {
        nSurfaceLayers 0;
    }
}
firstLayerThickness 5e-5;
expansionRatio 1.2;
minThickness 5e-7;
nGrow 0;
featureAngle 120;
slipFeatureAngle 30;
nRelaxIter 10;
nSmoothSurfaceNormals 1;
nSmoothNormals 10;
nSmoothThickness 10;
maxFaceThicknessRatio 0.7;
maxThicknessToMedialRatio 0.3;
minMedialAxisAngle 65;
nBufferCellsNoExtrude 0;
nLayerIter 100;
}

meshQualityControls
{
    maxNonOrtho 65;
    maxBoundarySkewness 4;
    maxInternalSkewness 2;
    maxConcave 80;
    minVol 1e-13;
    minTetQuality -1;
    minArea -1;
    minTwist 0.01;
    minDeterminant 1e-5;
    minFaceWeight 0.03;
    minVolRatio 0.01;
    minTriangleTwist -1;
    nSmoothScale 4;
    errorReduction 0.75;
    relaxed
    {
        maxNonOrtho 75;
    }
}
}
mergeTolerance 1e-6;

```

3.2.1 Refinement Surfaces and Regions

The propeller and wing were imported as triangulated STL files, while auxiliary geometries such as the AMI cylinder, refinement boxes, and the propeller wake cone were generated procedurally within **snappyHexMesh**.

Because the background mesh is relatively coarse, directly refining only the geometry surfaces would lead to abrupt cell-size jumps, often causing instability or divergence. To

avoid this, multiple refinement regions were introduced to provide a smooth transition between coarse and fine mesh zones.

The refinement strategy used in this mesh is as follows:

- **box1 and box2:** Reduce the background cell size (200 mm) to approximately 50 mm and 25 mm, respectively. Both boxes enclose the propeller and wing, providing a smoother progression in cell size.
- **AMI refinement region:** Refines cells inside the cylindrical AMI volume to approximately 6.25 mm.
- **propWake region:** Refines cells downstream of the propeller to 6.25 mm for proper wake and vortex resolution.
- **box3 region:** Nested within the AMI volume and enclosing the propeller itself; refines cell size further to approximately 3.125 mm for high resolution around the blades.

Surface-based refinement is applied using `refinementSurfaces`:

- **Propeller surface:** Final cell sizes in the range 0.39-0.78 mm to resolve blade edges.
- **Wing surface:** Final surface cell size around 1.56 mm.
- **AMI surface:** Cell size around 6.25 mm.

AMI surface handling: The AMI patch is defined as `faceType` `baffle`, which creates a two-sided internal surface. This generates two separate patches: `ami` and `ami_slave`. These are later merged into an Arbitrary Mesh Interface using `createPatch`. The `cellZone` keyword defines a rotating zone for the AMI region and the `cellZoneInside` keyword specifies whether the `cellZone` is region inside or outside of the AMI surface. This field is also essential for constructing Multi Reference Frame (MRF) steady-state alternatives.

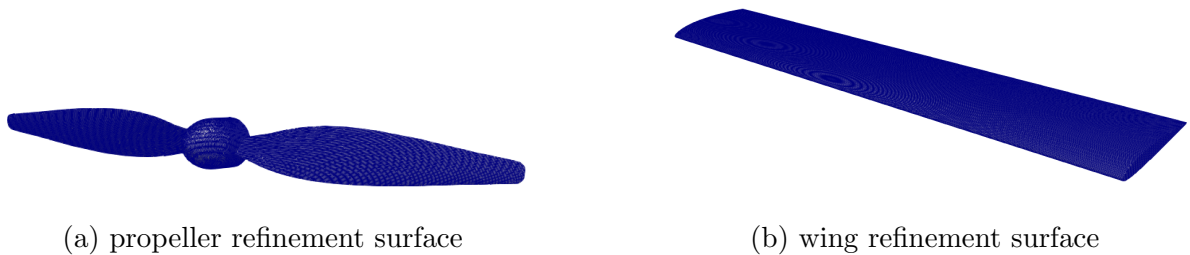


Figure 3.2: wing and propeller geometry used for this simulation

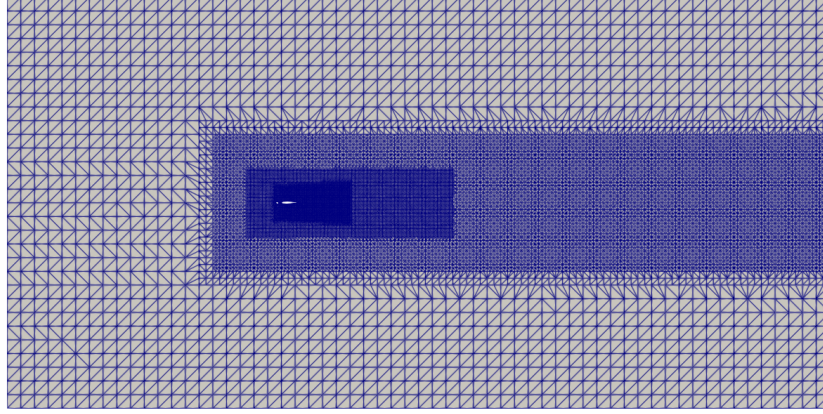


Figure 3.3: Mesh section showing refinement regions

3.2.2 Layer Addition

Boundary layer resolution near walls may be achieved either by:

- **Wall-resolved mesh** with $y^+ \approx 1$, or
- **Wall-function approach** where the first cell lies in the logarithmic region.

Wall resolved meshes are more accurate but significantly more expensive, as they require a very small first-layer thickness. In this case, a first-layer height of 5×10^{-5} m was chosen to target $y^+ \approx 1$ on both the wing and propeller surfaces.

The common issues that can be encountered in layer addition include:

- **Layer collapse** due to large curvature changes or insufficiently small adjacent cells.
- **Layer detachment** when expansion ratios are too high.
- **Insufficient layers being added** when the underlying surface mesh is too coarse, requiring either smaller base mesh cells or more layers.

If post-processing indicates that the resulting y^+ values deviate significantly from the target, the mesh must be regenerated with adjusted `firstLayerThickness` or increased surface refinement.

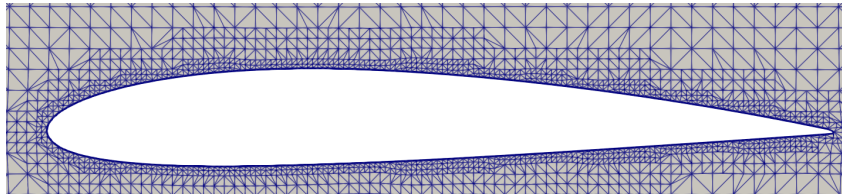


Figure 3.4: Mesh section showing added layers near wing surface

3.3 Creating AMI Patches

After mesh generation with `snappyHexMesh`, the AMI surface exists as a `faceZone` but must be converted into proper AMI patch pairs. The `createPatch` utility performs this conversion based on the `system/createPatchDict` configuration. Since the

faceZone was named `ami` in `snappyHexMeshDict`, `snappyHexMesh` automatically creates two patches: `ami` (master side) and `ami_slave` (slave side). These must be converted to `cyclicAMI` type patches.

3.3.1 createPatchDict Configuration

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "system";
    object       createPatchDict;
}

pointSync false;

patches
(
    {
        name          AMI1;
        patchInfo
        {
            type          cyclicAMI;
            matchTolerance 0.0001;
            neighbourPatch AMI2;
            transform      noOrdering;
            AMIMethod      faceAreaWeightAMI;
            lowWeightCorrection 0.2;
        }
        constructFrom patches;
        patches (ami);
    }

    {
        name          AMI2;
        patchInfo
        {
            type          cyclicAMI;
            matchTolerance 0.0001;
            neighbourPatch AMI1;
            transform      noOrdering;
            AMIMethod      faceAreaWeightAMI;
            lowWeightCorrection 0.2;
        }
        constructFrom patches;
        patches (ami_slave);
    }
);
```

The key parameters of `createPatchDict` are:

- **name:** New patch name (`AMI1` and `AMI2` replace `ami` and `ami_slave`)
- **type:** Must be `cyclicAMI` to enable AMI functionality

- **matchTolerance**: Geometric tolerance for face overlap detection (default: 0.0001). Smaller values require more precise overlap
- **neighbourPatch**: Name of the corresponding AMI patch. Each patch must reference the other
- **transform**: Set to `noOrdering` for arbitrary (non-periodic) interfaces
- **AMIMethod**: Interpolation method. `faceAreaWeightAMI` uses area-weighted averaging based on face overlap areas
- **lowWeightCorrection**: Threshold for low-weight correction (0.0-1.0). When overlap is poor (weight sum < 1.0), this parameter enables extrapolation to improve stability. Values of 0.2-0.4 are typical
- **constructFrom**: Source for patch creation. `patches` indicates construction from existing patches
- **patches**: List of source patches to convert
- **pointSync**: When `false`, prevents point field synchronization across patches (appropriate for AMI)

3.3.2 Running createPatch

The utility to create AMI patches is executed as:

```
createPatch -overwrite
```

The `-overwrite` flag modifies the mesh in place. After execution, The patches must exist in `constant/polyMesh/boundary` with `cyclicAMI` type.

3.4 Mesh Quality Checks

The `checkMesh` utility is used to evaluate and validate the quality of the generated mesh.

```
checkMesh > log.checkMesh
```

Running above command provides quick essential diagnostics about mesh health, including key metrics such as aspect ratio, skewness, and non-orthogonality. These are among the most important parameters influencing solver stability and accuracy.

Aspect ratio:

Measures the ratio between the longest and shortest edges of a cell. Ideally, this should be close to 1 for internal cells. However, boundary-layer cells may legitimately have large aspect ratios (e.g. up to 100 or more) due to stretching normal to the wall.

Skewness:

Quantifies how far a cell deviates from an ideal shape (e.g. a regular hexahedron). Skewness close to 0 is ideal. In most OpenFOAM solvers, skewness below 4 is generally acceptable, although lower is always preferred.

Non-orthogonality:

Represents the angle between a face normal and the vector connecting adjacent cell centers. A perfectly orthogonal mesh has non-orthogonality equal to 0° . In practice, maximum non-orthogonality $< 65^\circ$ is considered good, values up to 70° to 75° are manageable with appropriate numerical schemes (e.g. limited linear), whereas the values above 80° can cause solver instability.

For a more detailed analysis, including checks of mesh topology and geometry consistency, the following flags are used:

```
checkMesh -allGeometry -allTopology > log.checkMesh
```

This extended output includes additional diagnostics such as:

- face area and volume checks,
- concave/convex cell checks,
- min/max determinant,
- illegal faces and cells,
- check for open or non-manifold edges,
- check for highly warped faces.

While mesh quality should be kept as high as possible, acceptable thresholds also depend on the nature of the flow, turbulence modeling approach, numerical schemes, and solver robustness. High-fidelity simulations typically require stricter mesh quality criteria.

The relatively high count of concave cells and cells with small determinant here is

Parameter	Value	Assessment
Max skewness	3.99767	Acceptable (< 4)
Max non-orthogonality	65.0107°	Within limits ($< 70^\circ$)
Average non-orthogonality	7.02773°	Good ($< 15^\circ$)
Max aspect ratio	53.9146	Acceptable (boundary layer)
Mesh Quality Issues		
Concave cells	133,588 (2.67% of total)	Expected for complex geometry
Cells with small determinant	59,453 (1.19% of total)	Localized near blade edges
Concave faces	1,656	Acceptable
Warped faces	77	Minimal

Table 3.1: Critical mesh quality metrics for propeller wing case

the characteristic of propeller blade meshing, particularly near the blade leading/trailing

edges, tip regions, and hub junctions where geometry complexity is highest. These quality issues are localized and represent only 2.67% and 1.19% of the total cell count respectively. The critical parameters for solver stability (max non-orthogonality and skewness) remain within acceptable limits. For complex rotating applications, such localized quality degradation is often unavoidable and acceptable, provided the most of the mesh maintains good quality and appropriate numerical schemes and robust solvers are employed.

Chapter 4

MRF Setup

4.1 Multi Reference Frame

Multi Reference Frame helps to setup a steady state case of mesh having rotating bodies. Unlike AMI which actually moves the mesh, MRF keeps the mesh stationary but modifies the governing equations in the rotating region to account for rotational effects. The computational domain is divided into two zones: a rotating zone and a stationary zone.

- **Stationary zone:** Inertial reference frame where standard Navier-Stokes equations apply
- **Rotating zone:** Non-inertial rotating reference frame where modified equations account for rotation

In the rotating zone, additional source terms are added to the momentum equations to account for centrifugal force and coriolis force corresponding to the rotation.

- **Centrifugal force:** $\vec{F}_{\text{centrifugal}} = \rho \vec{\omega} \times (\vec{\omega} \times \vec{r})$
- **Coriolis force:** $\vec{F}_{\text{Coriolis}} = -2\rho \vec{\omega} \times \vec{U}_{\text{rel}}$

where $\vec{\omega}$ is the angular velocity vector, $\vec{r}$ is the position vector from the rotation axis, and $\vec{U}_{\text{rel}}$ is the velocity relative to the rotating frame.

The rotating zone is 'frozen' meaning the cellzone in mesh doesn't rotate. So,

- The mesh geometry doesn't change.
- Flow patterns do not evolve in time.
- Only steady-state and time-averaged performance results are obtained.
- Transient effects like propeller blade rotating and vortex shedding are not captured.

4.1.1 MRFProperties configuration

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
```

```

    location      "constant";
    object        MRFProperties;
}
MRF1
{
    cellZone      rotor;
    active         yes;
    nonRotatingPatches (AMI1 AMI2);
    origin        (-0.05 0 0.54);
    axis          (1 0 0);
    omega         733.04;  \\rad/s
}

```

The key parameters of **MRFProperties** are:

- **cellZone**: Name of the rotating region defined using **topoSet** or **snappyHexMesh**. This zone undergoes MRF treatment.
- **active**: Enables/disables MRF for this zone. Set to **yes** to activate
- **nonRotatingPatches**: List of patches within the MRF zone that should be treated as stationary (e.g., hub surfaces, shaft). Critical when AMI patches are present, as they must remain stationary to maintain interface integrity
- **origin**: Cartesian coordinates of a point on the rotation axis (propeller hub center: $-0.05, 0, 0.54$ m)
- **axis**: Unit vector defining rotation axis direction using right-hand rule ($(1, 0, 0)$ indicates rotation about x-axis)
- **omega**: Constant angular velocity in rad/s. Here, $733.04 \text{ rad/s} \approx 7000 \text{ RPM}$

While cases with MRF cannot capture the unsteady behavior, they can be quick and better method to study the approximate steady-state behavior and time-averaged performance which can be enough for many technical applications. Before starting unsteady rotating mesh simulation, it is a good practice to begin with steady-state simulation with MRF and compare the time-averaged results with each-other.

Chapter 5

Dynamic Mesh and AMI Setup

5.1 Arbitrary Mesh Interface

AMI (Arbitrary Mesh Interface) is a technique in OpenFOAM that allows two non-conformal mesh regions to communicate and exchange information, enabling simulation of relative motion without remeshing. AMI connects two mesh patches that:

- Don't share the same nodes.
- Can have different cell sizes and layouts.
- Can move relative to each other.
- Are topologically different but geometrically overlapping.

AMI calculates the overlap area between faces on opposite sides of the interface. For each face on the master patch, it finds which slave patch faces it overlaps with and creates weighting factors based on overlap areas. When transferring data (velocity, pressure, etc.) across the interface, the value on master faces is calculated as:

$$\phi_{\text{master},i} = \sum_j w_{ij} \cdot \phi_{\text{slave},j} \quad (5.1)$$

where w_{ij} are weights proportional to overlap areas and satisfy $\sum_j w_{ij} = 1$. This ensures conservation of flux across the interface. The weights update automatically as patches rotate or move relative to each other. Weights are proportional to overlap areas and it ensures the conservation of flux across the interface. It updates automatically as patches rotate or move relative to each other. Mass, momentum, and energy are conserved across the interface. It uses area-weighted averaging to interpolate values.

5.2 dynamicMeshDict Configuration

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
```

```

    object      dynamicMeshDict;
}
dynamicFvMesh   dynamicMotionSolverFvMesh;
motionSolverLibs (fvMotionSolvers);
motionSolver     solidBody;
cellZone         rotor;
solidBodyMotionFunction rotatingMotion;
origin   (-0.05 0 0.54);
axis     (1 0 0);
omega    table
(
    (0 0)
    (0.05 733.04)
    (1 733.04)
);
predictorCorrector true;
report true;

```

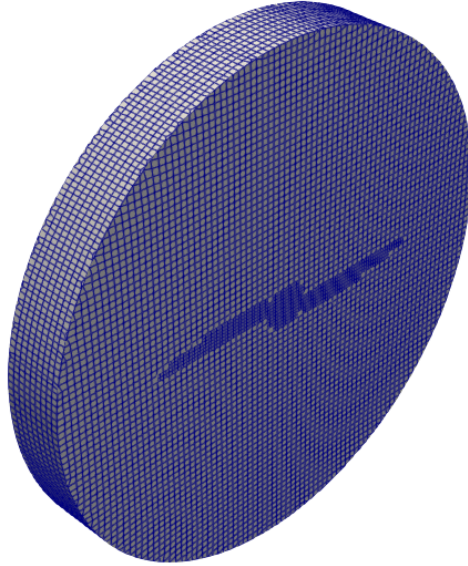
The key parameters of `dynamicMeshDict` are:

- **dynamicFvMesh:** Specifies the dynamic mesh type. `dynamicMotionSolverFvMesh` indicates mesh motion is governed by a motion solver as opposed to topology changes.
- **motionSolverLibs:** Library containing motion solver functions. `fvMotionSolvers` provides standard motion solvers.
- **motionSolver:** Type of motion solver. `solidBody` means rigid body motion without mesh deformation.
- **cellZone:** Name of the rotating mesh region defined using `topoSet` or `snappyHexMesh`.
- **solidBodyMotionFunction:** Specifies the motion type. `rotatingMotion` defines rotational motion about an axis.
- **origin:** Point on rotation axis of the cellZone (propeller hub center here).
- **axis:** Rotation direction using right-hand rule.
- **omega:** Angular velocity schedule in rad/s (733.04 rad/s). Gradual ramp-up (0-0.05s) prevents numerical instability.

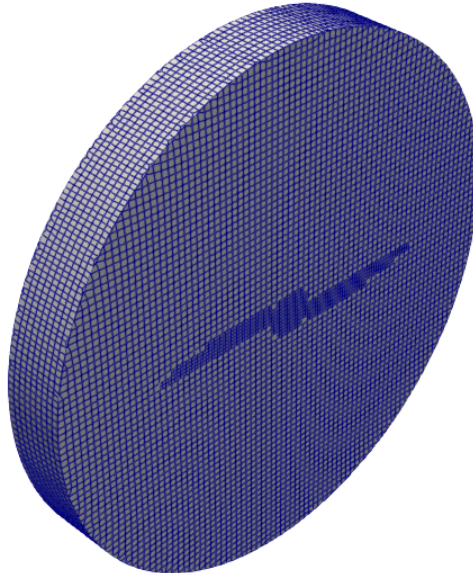
The above given setup of the `dynamicMeshDict` file sets up an unsteady simulation here. Before running the full simulation, the dynamic mesh setup can be verified using the following command:

```
moveDynamicMesh -noFunctionObjects
```

This command runs only the motion solver without the CFD solver, providing a quick check of mesh motion behavior. If the AMI weights are too small or too large than 1, the ami surfaces require further refinement or the cylinders might require proper alignment. It is crucial to have AMI weights as close to 1 as possible.



(a) mesh of AMI1 patch



(b) mesh of AMI2 patch

Figure 5.1: Mesh showing AMI1 and AMI2 patch together. They should look very similar

Chapter 6

Boundary and Initial Conditions

6.1 Case Initialization

This simulation is configured for incompressible flow. Therefore, the solver resolves the momentum, continuity, and turbulence model equations. The Spalart-Allmaras (SA) turbulence model is selected for this analysis. SA is a one-equation model widely adopted in external aerodynamics RANS and URANS simulation, particularly where the flow remains largely attached. It is favored for its numerical stability and computational efficiency compared to two-equation models. While the k - ω SST model offers higher fidelity for complex flow separation, it is computationally more expensive and can exhibit sensitivity to initialization. Given the aerodynamic nature of the wing-propeller interaction, SA provides an optimal balance of accuracy and cost. If the interest of study is flow separation, the k - ω SST model can be a better choice. For this incompressible case, the transport properties and four primary field files are initialized: velocity (U), kinematic pressure (p), turbulent viscosity (ν_t), and the modified turbulent viscosity ($\tilde{\nu}$).

6.1.1 Velocity Field (U)

The velocity field utilizes the movingWallVelocity condition for the propeller to account for the frame of reference or mesh motion, while the wing utilizes a standard no-slip condition. Prescribed velocity is defined in the inlet whereas the outlet specifies inletOutlet condition which can prevent numerical instabilities from backflow at outlet.

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        volVectorField;
    location     "0";
    object       U;
}
#include        "include/initialConditions"
dimensions     [0 1 -1 0 0 0 0];
internalField  uniform $UInf;
boundaryField
{
    inlet
    {
        type          fixedValue;
        value          uniform $UInf;
```

```

}
outlet
{
    type            inletOutlet;
    inletValue      $internalField;
    value           $internalField;
}
farField
{
    type            zeroGradient;
}
propeller
{
    type            movingWallVelocity;
    value           uniform (0 0 0);
}
wing
{
    type            noSlip;
}
symmetry
{
    type            symmetryPlane;
}
AMI1
{
    type            cyclicAMI;
    value           uniform $UInf;
}
AMI2
{
    type            cyclicAMI;
    value           uniform $UInf;
}
}

```

6.1.2 Pressure Field (p)

Since the solver is incompressible, p represents kinematic pressure (P/ρ) with dimensions m^2/s^2 .

```

FoamFile
{
    version         2.0;
    format           ascii;
    class            volScalarField;
    location         "0";
    object           p;
}
#include            "include/initialConditions"
dimensions          [0 2 -2 0 0 0 0];
internalField       uniform $pInf;
boundaryField
{
    inlet
    {
        type            zeroGradient;
    }
}

```

```

    }
    outlet
    {
        type            fixedValue;
        value            uniform $pInf;
    }
    wall
    {
        type            zeroGradient;
    }
    farField
    {
        type            zeroGradient;
    }
    symmetry
    {
        type            symmetryPlane;
    }
    AMI1
    {
        type            cyclicAMI;
        value            uniform $pInf;
    }
    AMI2
    {
        type            cyclicAMI;
        value            uniform $pInf;
    }
}

```

A reference gauge kinematic pressure is specified at the outlet whereas the zeroGradient is applied on the wall and farField.

6.1.3 Turbulence Fields

The Spalart-Allmaras model requires the solution of $\tilde{\nu}$ (nuTilda), while ν_t (nut) is calculated algebraically though it must be initialized.

```

FoamFile
{
    version            2.0;
    format              ascii;
    class               volScalarField;
    location            "0";
    object              nut;
}
#include               "include/initialConditions"
dimensions             [0 2 -1 0 0 0 0];
internalField          uniform $nutVal;
boundaryField
{
    inlet
    {
        type            calculated;
        value            uniform $nutVal;
    }
    outlet

```

```

{
    type                zeroGradient;
}
wall
{
    type                nutUSpaldingWallFunction;
    value               uniform 0;
}
farField
{
    type                zeroGradient;
}
symmetry
{
    type                symmetryPlane;
}
AMI1
{
    type                cyclicAMI;
    value               uniform $nutVal;
}
AMI2
{
    type                cyclicAMI;
    value               uniform $nutVal;
}
}

```

```

FoamFile
{
    version             2.0;
    format               ascii;
    class                volScalarField;
    location             "0";
    object               nuTilda;
}
#include                "include/initialConditions"
dimensions              [0 2 -1 0 0 0 0];
internalField            uniform $nuTildaVal;
boundaryField
{
    inlet
    {
        type            fixedValue;
        value            uniform $nuTildaVal;
    }
    outlet
    {
        type            zeroGradient;
    }
    wall
    {
        type            fixedValue;
        value            uniform 0;
    }
    farField
    {
        type            zeroGradient;
    }
}

```

```

    }
    symmetry
    {
        type                symmetryPlane;
    }
    AMI1
    {
        type                cyclicAMI;
        value                uniform $nuTildaVal;
    }
    AMI2
    {
        type                cyclicAMI;
        value                uniform $nuTildaVal;
    }
}

```

The nutUSpaldingWallFunction implements Spalding's law of the wall. It blends smoothly from viscous sublayer through buffer layer to log-layer and is more accurate than standard wall functions for adverse pressure gradients. Though it requires $y^+ \approx 30$ -300 for optimal accuracy, $y^+ \geq 1$ or $y^+ \geq 5$ with refined mesh can also work but increases computational cost.

6.2 Properties and Constants

Global variables are defined in the initialConditions file to ensure consistency across all boundary files. This approach makes the case easier to setup for parametric studies and reduces the risk of inconsistent boundary specifications.

```

FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "0/include";
    object       initialConditions;
}
//Ambient air variables
UInf           (20 0 0); // m/s
pInf           0; // m^2/s^2 (p/rho)
rhoInf         1.225; // kg/m^3
muVal          1.82e-5; // kg/(m.s)
nuVal          1.5e-5; // m^2/s

//spalartAllmaras turbulence model
nuTildaVal     4.5e-5; // m^2/s nuTilda = 3*nu to 5*nu
nutVal         3.15e-6; // m^2/s nut = 0.07*nuTilda

```

The turbulence model and transport properties are defined as follow in the constant directory:

```

FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
}

```

```

        location    "constant";
        object      turbulenceProperties;
    }
    simulationType  RAS;
    RAS
    {
        RASModel      SpalartAllmaras;
        turbulence     on;
        printCoeffs    on;
    }

```

```

FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
    object       transportProperties;
}
transportModel  Newtonian;
nu              1.5e-05;

```

The Newtonian transport model assumes constant viscosity valid for air at moderate temperatures. The Kinematic viscosity $1.5 \times 10^{-5} m^2/s$ corresponds to air at $\approx 15^\circ - 25^\circ C$. For non-Newtonian fluids or temperature-dependent simulations, alternative models are available

Chapter 7

Solver Configuration and Controls

The solver configuration and controls are set in system directory of the case setup. The `controlDict` defines the solver and solver control parameters. For a steady state simulation, `simpleFoam` is used whereas for the transient simulation, `pimpleFoam` is utilized.

7.1 Steady state Simulation

The SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) algorithm is a widely-used iterative procedure for solving incompressible flow problems. `SimpleFoam` implements this algorithm for steady-state simulations where the solution is advanced pseudo-temporally until convergence.

7.1.1 `controlDict` configuration

```
FoamFile
{
    version 2.0;
    format ascii;
    class dictionary;
    location "system";
    object controlDict;
}
application simpleFoam;
startFrom latestTime;
startTime 0;
stopAt endTime;
endTime 1000;
deltaT 1;
writeControl timeStep;
writeInterval 50;
purgeWrite 0;
writeFormat ascii;
writePrecision 6;
writeCompression no;
timeFormat general;
timePrecision 6;
runTimeModifiable yes;
adjustTimeStep no;
```

The key parameters of `controlDict` for steady-state case are:

- **application**: Specifies `simpleFoam` solver for steady-state incompressible flow.
- **startFrom**: Specifies time from which simulation should start. `latestTime` Continues from the last saved time directory.
- **deltaT**: Dummy time step representing one iteration.
- **endTime**: Total number of iterations (1000 iterations in this case).
- **writeInterval**: Saves solution every 50 iterations here.
- **purgeWrite**: Keeps all saved time directories here. It is used to save disk space.
- **runTimeModifiable**: Allows editing `controlDict` while solver is running if set yes or true.
- **adjustTimeStep**: Fixed iteration step.

The propeller-wing case should converge within 500 to 1500 iterations depending on geometry complexity and mesh quality.

7.2 Unsteady Simulation

The PIMPLE algorithm combines PISO (Pressure-Implicit with Splitting of Operators) and SIMPLE methods, enabling larger time steps while maintaining stability. This is essential for capturing transient phenomena such as propeller blade passage, vortex shedding, and unsteady wake interactions.

7.2.1 controlDict configuration

```
FoamFile
{
  version 2.0;
  format ascii;
  class dictionary;
  location "system";
  object controlDict;
}
application pimpleFoam;
startFrom latestTime;
startTime 0;
stopAt endTime;
endTime 1;
deltaT 0.00001;
writeControl adjustable;
writeInterval 0.002;
purgeWrite 0;
writeFormat ascii;
writePrecision 6;
writeCompression no;
timeFormat general;
timePrecision 6;
runTimeModifiable yes;
adjustTimeStep yes;
```

```
maxCo    5;
maxDeltaT 5e-5;
```

While most of the key parameters of `controlDict` are same for steady-state and transient case, few differences are pointed here:

- **deltaT**: Specifies Initial physical time step (1e-5s here)
- **maxCo**: Sets maximum allowable Courant number (5 here).
- **writeInterval**: Saves solution every 2e-5s here.
- **maxDeltaT**: Sets upper limit on time step to maintain temporal resolution(5e-5s here).
- **adjustTimeStep**: Allows automatic time step adjustment based on Courant number if set to yes.

At the rotation speed of 7000RPM, it takes 2.38e-5 sec to rotate by 1°. The smaller deltaT was used to capture detailed flow features.

7.3 Discretization Schemes

The fvSchemes dictionary specifies how spatial and temporal derivatives are discretized. The choice of schemes significantly affects solution accuracy, stability, and computational cost.

7.3.1 fvSchemes configuration

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "system";
    object       fvSchemes;
}
ddtSchemes
{
    default      steadyState; \\comment this line for pimpleFoam
    \\default    Euler; \\uncomment this line for pimpleFoam
}
gradSchemes
{
    default      Gauss linear;
    grad(p)      Gauss linear;
    grad(U)      cellLimited Gauss linear 1;
}
divSchemes
{
    default      none;
    div(phi,U)   bounded Gauss linearUpwind limited;
    div(phi,nuTilda)   bounded Gauss linearUpwind limited;
    div(phid,p)  Gauss upwind;
```

```

    div(phiMRF,p)    Gauss upwind; \\remove this for pimpleFoam
    div(phiMRF,U)    Gauss upwind; \\remove this for pimpleFoam
}
laplacianSchemes
{
    default          Gauss linear limited corrected 0.33;
}
interpolationSchemes
{
    default          linear;
}
snGradSchemes
{
    default          limited corrected 0.33;
}
wallDist
{
    method    meshWave;
}

```

Various schemes used and there purposes are:

Time Derivative (ddtSchemes):

- **steadyState**: It sets time derivative terms to zero for SIMPLE iterations.
- **Euler**: First-order implicit, unconditionally stable, less accurate than backward.
- **backward**: Second-order implicit, better accuracy but requires smaller time steps.

Gradient (gradSchemes):

- **Gauss linear**: Second-order accurate central differencing.
- **cellLimited**: Prevents unbounded gradients in regions of high velocity gradients.

Divergence (divSchemes):

- **linearUpwind**: Second-order upwind scheme, captures advection accurately.
- **bounded**: It ensures transported quantities remain within physical bounds.
- **limited**: It blends to first-order upwind in regions of high gradients.

Laplacian (laplacianSchemes):

- **limited corrected 0.33**: Non-orthogonal mesh correction with limiting. It is crucial for meshes with high non-orthogonality.

7.4 Linear Solvers

The fvSolution dictionary specifies linear equation solvers, tolerances, and algorithm controls. Proper configuration is critical for convergence and computational efficiency.

7.4.1 fvSolution configuration

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       fvSolution;
}
solvers
{
    "pcorr.*"
    {
        solver          GAMG;
        tolerance        1e-2;
        relTol           0;
        smoother         DICGaussSeidel;
        cacheAgglomeration no;
        maxIter           50;
    }
    p
    {
        $pcorr;
        tolerance        1e-6;
        relTol           0.01;
    }
    pFinal
    {
        $p;
        tolerance        1e-6;
        relTol           0;
    }
    "(U|nuTilda)"
    {
        solver          smoothSolver;
        smoother         symGaussSeidel;
        tolerance        1e-6;
        relTol           0.1;
    }
    "(U|nuTilda)Final"
    {
        solver          smoothSolver;
        smoother         symGaussSeidel;
        tolerance        1e-6;
        relTol           0;
    }
}
cache
{
    grad(U);
}
```

Geometric-Algebraic Multi-Grid (GAMG):

- It is optimal for solving pressure equations efficiently using multi-level hierarchy.
- Its solution time scales nearly linearly with mesh size.

- The `DICGaussSeidel` smoother provides robust smoothing for symmetric positive definite matrices.

Smooth Solver (`smoothSolver`):

- It is efficient for momentum and turbulence transport equations.
- It has lower computational cost than GAMG for non-elliptic equations.
- It doesn't require additional storage for coarse grid levels.
- `symGaussSeidel` provides symmetric iteration with good convergence.

7.4.2 PIMPLE Controls

This is to be added in `fvSolution` at the end, if `pimpleFoam` is the selected solver.

```
PIMPLE
{
    momentumPredictor    yes;
    nOuterCorrectors      4;
    nCorrectors           2;
    nNonOrthogonalCorrectors 2;
    residualControl
    {
        P
        {
            tolerance     1e-5;
            relTol         0;
        }
        U
        {
            tolerance     1e-5;
            relTol         0;
        }
        "nuTilda"
        {
            tolerance     1e-5;
            relTol         0;
        }
    }
}
```

The key parameters for controlling linear solvers for `pimpleFoam` are:

- **momentumPredictor:** Solves momentum equation before pressure correction (improves convergence) if set to yes.
- **nOuterCorrectors:** defines number of PIMPLE loops per time step (balances accuracy and cost) (4 here).
- **nCorrectors:** defines pressure-velocity corrections per outer loop (PISO correctors) (2 here).
- **nNonOrthogonalCorrectors:** defines additional corrections for non-orthogonal meshes (2 here).

The value of `nOuterCorrectors` should be increased for larger time steps or complex geometries and should be reduced for well resolved meshes and smaller time steps.

7.4.3 SIMPLE Controls

This is to be added in `fvSolution` at the end, if `simpleFoam` is the selected solver.

```
SIMPLE
{
    nNonOrthogonalCorrectors 1;
    residualControl
    {
        p          1e-5;
        U          1e-5;
        "nuTilda"   1e-5;
    }
}
relaxationFactors
{
    fields
    {
        p 0.3 ;
    }
    equations
    {
        U 0.7 ;
        nuTilda 0.7 ;
    }
}
```

The residual control sets up the convergence criteria for the solvers. The simulation terminates when all field residuals drop below specified tolerances. Forces or force coefficients should also stabilize if the simulation has converged. The relaxation factors defines the relaxation for various field. The under-relaxation stabilizes the iterative solution by limiting the changes between iterations. Lower values of relaxation is more stable but converges slowly whereas the higher values allows faster change but may cause oscillations.

Chapter 8

Function Objects and Data Extraction

8.1 Force and Moment Extraction

Function objects are powerful OpenFOAM tools that execute during simulation runtime to extract data, perform calculations, or modify the flow field. They are essential for monitoring convergence, extracting aerodynamic forces, calculating performance data, and post-processing flow features. Following function objects were defined to extract crucial data like forces from propeller and wing, wall shear stresses, y^+ values, AMI weights, Q criterion, etc:

```
functions
{
    #includeFunc Q
    #include "AMIWeights"
    #includeFunc solverInfo
    yPlus
    {
        type        yPlus;
        libs        (fieldFunctionObjects);
        executeControl writeTime;
        writeControl  writeTime;
        patches      (propeller wing);
    }
    wallShearStress1
    {
        type        wallShearStress;
        libs        ("libfieldFunctionObjects.so");
        patches      (propeller wing);
        executeControl writeTime;
        writeControl  writeTime;
    }
    propForce
    {
        type                    forces;
        libs                    ("libforces.so");
        writeControl            timeStep;
        timeInterval            0.001;
        log                      true;
        patches                  ("propeller");
        rho                      rhoInf;
    }
}
```

```

        rhoInf          1.225;
        pRef            0;
        CofR            (-0.05 0 0.54);
        pitchAxis       (1 0 0);
    }
    wingForce
    {
        type            forces;
        libs            ("libforces.so");
        writeControl     timeStep;
        timeInterval     0.001;
        log              true;
        patches          ("wing");
        rho              rhoInf;
        rhoInf          1.225;
        pRef            0;
        CofR            (0.059 0 0.725);
        pitchAxis       (0 0 1);
    }
}

```

Various schemes used and there purposes are:

Q-criterion

The Q-criterion function helps to identify vortex structures in the flow. It is mathematically given as:

$$Q = \frac{1}{2} \cdot (\Omega^2 - S^2) \quad (8.1)$$

where Ω is vorticity magnitude, S is strain rate magnitude. The positive Q region indicates vortex cores and is useful for visualizing propeller tip vortices and wake structures.

solverInfo

The solver info function outputs detailed solver performance metrics and residuals data. It can be useful for comparing two different configurations for same setup.

Divergence (divSchemes):

The yPlus function object calculates the dimensionless wall distance which is crucial for the performance of turbulence model and convergence of simulation. It is mathematically given as:

$$y^+ = \frac{u_\tau \cdot y}{\nu} \quad (8.2)$$

where u_τ is friction velocity, y is wall distance, ν is kinematic viscosity.

Wall Shear Stress

The Wall shear stress (τ_w) is critical for understanding boundary layer behavior and drag prediction. It is mathematically given as:

$$\tau_w = \mu \left. \frac{\partial u}{\partial y} \right|_{\text{wall}} \quad (8.3)$$

where u_τ is friction velocity, y is wall distance, ν is kinematic viscosity. High shear stress regions indicate flow acceleration, attached boundary layers, and potential separation points when approaching zero.

forces

The forces function object calculates forces exerted due to pressure and stress distribution on the wall surfaces. The pitch axis defines the axis about which the rotation of body is expected and the center of rotation (CofR) specifies the point about which the body rotates. After the force and moment vector for the wing is obtained, it can be resolved to obtained lift, drag, and moment. From the force and moment vector for the propeller, thrust, torque, and power can be obtained. Using them, non-dimensional coefficients like thrust coefficient(C_T), power coefficient(C_P), advance ratio(J), and propeller efficiency(η) can be obtained mathematically:

$$C_T = \frac{T}{\rho \cdot n^2 \cdot D^4} \quad (8.4)$$

$$C_P = \frac{P}{\rho \cdot n^3 \cdot D^5} \quad (8.5)$$

$$J = \frac{V}{n \cdot D} \quad (8.6)$$

$$\eta = \frac{T \cdot V}{P} \quad (8.7)$$

where n is angular frequency, D is propeller diameter, and V is freestream velocity.

8.2 Monitoring the execution

The following parameters should be monitored continuously to ensure the case runs successfully:

- Residuals: These should tend to zero and should not oscillate too much for the simulation to converge.
- AMI interpolation weights: These must remain very close to 1 and getting towards zero should be avoided.
- Courant number and time step stability: These must remain bounded. The Courant number should not increase to very large number and the time step should not decrease to very small number.
- Forces: These must approach towards a finite value and should not oscillate too much.

Chapter 9

Appendices

9.1 mesh generation script

```
#!/bin/bash
blockMesh
surfaceFeatureExtract
decomposePar -force
mpirun -np 6 snappyHexMesh -overwrite -parallel | tee log.snappyHexMesh
rm -rf 0 && cp -r 0.orig 0
reconstructParMesh -constant | tee log.reconstructParMesh
rm -rf processor*
createPatch -overwrite
renumberMesh -overwrite | tee log.renumberMesh
checkMesh -allGeometry -allTopology | tee log.checkMesh
```

9.2 steady state run script

```
decomposePar
mpirun -np 6 simpleFoam -parallel | tee log.simpleFoam
reconstructPar | tee log.reconstructPar
```

9.3 unsteady run script

```
decomposePar
mpirun -np 6 pimpleFoam -parallel | tee log.pimpleFoam
reconstructPar | tee log.reconstructPar
```